

Stacks, Queues & the Public/Private Divide

Quiz 01 — Instructor key (omit from print run)

array[0] Class Syntax

model answer

Two `class` definitions, data members only. Access specifiers are not required by the prompt, but `private:` members are the expected default and should be rewarded.

// STACK — ARRAY-BASED

```
class Stack {
private:
    int *array;    // heap-allocated element buffer
    int  capacity; // number of slots in array
    int  top;      // index of the most recent push; -1 when empty
};
```

// QUEUE — SINGLY LINKED LIST

```
class Queue {
private:
    Node *front; // dequeue from here (head)
    Node *rear;  // enqueue here (tail)
};
```

GRADING NOTES

What counts

Stack — min	<code>int *array;</code> + <code>int top;</code> . Full credit also for a <code>capacity</code> / <code>size</code> member; templated element type is fine.
Stack — watch	Missing the <code>*</code> on <code>array</code> ; <code>int array[];</code> with no size (not a valid non-static member); using <code>front</code> / <code>rear</code> names.
Queue — min	<code>Node *front;</code> + <code>Node *rear;</code> . A <code>Node</code> definition is not required; an extra <code>int count;</code> is acceptable.
Queue — watch	Only one pointer (no O(1) enqueue without <code>rear</code>); calling the members <code>top</code> / <code>array</code> ; pointer type written as the value type (<code>Node front;</code>).
Auto-deductions	Any method (<code>push</code> , <code>pop</code> , <code>enqueue</code> , <code>dequeue</code>), constructor, or destructor — these were explicitly excluded.
Cosmetic	<code>Node *front</code> vs <code>Node* front</code> — spacing does not matter. Trailing semicolon after <code>}</code> required for the class to compile.
If Node shown	Should carry <code>int data;</code> and <code>Node *next;</code> .